

Um perfil UML para o paradigma orientado a notificações (PON)

Luciana V. B. Wiecheteck¹, Paulo C. Stadzisz¹, Jean M. Simão¹

lucianavbw@gmail.com, stadzisz, jeansimao {@utfpr.edu.br}

Universidade Tecnológica Federal do Paraná - UTFPR

¹ Programa de Pós-Graduação em Engenharia Elétrica e Informática Industrial - CPGEI

Resumo: Este artigo apresenta uma proposta de perfil UML para a modelagem de software utilizando o Paradigma Orientado a Notificações (PON). A UML possui mecanismos de extensão que permitem a adição de novas sintaxe e semântica aos seus elementos visando a modelagem de domínios particulares de aplicação. Um conjunto desses mecanismos de extensão agrupados dentro de um pacote UML é denominado Perfil UML. Por sua vez, o PON se apresenta como uma alternativa aos Paradigmas de Programação Imperativa (PI), incluindo o Paradigma Orientado a Objetos (POO), e aos Paradigmas de Programação Declarativa (PD), propondo-se a eliminar deficiências destes nos aspectos de redundâncias e acoplamento de avaliações causais que impactam no desempenho e paralelismo/distribuição de aplicações. Este novo paradigma tem sido materializado em termos de programação, mas não possuía ainda um método formalizado para orientar os desenvolvedores na elaboração de projetos de software. Portanto, o perfil proposto define um primeiro passo na criação deste método: formalizar os principais conceitos do PON por meio da utilização de mecanismos de extensão - como estereótipos, valores etiquetados e restrições - que melhor adequam a sua modelagem em UML. A aplicação do perfil UML é ilustrada com exemplos envolvendo a modelagem de um Simulador de Portão Eletrônico. Os resultados demonstram que o perfil criado possui grande valia na definição de programas em PON e visa sua aplicação em processos de projetos de software que utilizam este paradigma de programação.

Palavras chave: Modelagem de Software, Perfil UML, Paradigma Orientado a Notificações (PON)

Abstract: This article presents a UML Profile for software modeling using the Notification Oriented Paradigm (NOP). UML has extension mechanisms that allow the addition of new syntax and semantic to its elements aiming at software modeling for specific domain applications. A set of these extension mechanisms grouped into a UML package is named UML Profile. NOP presents itself as an alternative to the Imperative Programming (IP) paradigms, such as Object Oriented Paradigm (OOP), as well as to the Declarative Programming (DP) paradigms, with the purpose of eliminating deficiencies of those paradigms concerning to redundancy issues and coupling of causal expressions, which affect the execution performance and parallelism/distribution of applications. This paradigm has already been consolidated in terms of programming but did not possess a formalized method for software design. Thus, the proposed profile defines a first step in the definition of this method: to formalize the main concepts of this paradigm by means of extension mechanisms usage - as stereotypes, tagged values and constraints - that better fit its modeling in the UML. The proposed UML Profile usage is illustrated by means of a modeling example of an electronic gate application. The results showed that the proposed UML profile has an important value in the modeling of NOP programs and can be used in software design processes that use this new programming paradigm.

Keywords: Software Modeling, UML Profile, Notification Oriented Paradigm (NOP)

1. Introdução

A capacidade de processamento computacional tem crescido em função da evolução das tecnologias neste contexto [Tanenbaum e Van Steen, 2002]. Entretanto, recursos oferecidos por soluções computacionais modernas, tais como paralelismo e distribuição ou mesmo a utilização da capacidade plena de cada processador, nem sempre são devidamente aproveitados em função de limitações das técnicas de programação [Simão e Stadzisz, 2008, 2009].

Na verdade, técnicas de programação baseadas no estado da arte, como o chamado Paradigma de Programação Orientada a Objetos (POO) ou Sistema Baseados em Regras (SBR), sofrem de limitações intrínsecas de seus paradigmas. Estes paradigmas poderiam ser genericamente classificados como Paradigma Imperativo (PI) e Paradigma Declarativo (PD) que englobam respectivamente o POO e os SBR [Banaszewski, 2009].

Particularmente, estes paradigmas levam ao forte acoplamento de expressões causais e redundâncias

decorrentes das suas avaliações. Estas limitações dificultam a execução paralela ou distribuída de programas e frequentemente comprometem o seu desempenho pleno mesmo em sistemas monoprocessados. Assim, existem motivações para buscas de alternativas aos PI e PD, com o objetivo de eliminar ou diminuir as desvantagens deles [Banaszewski et al., 2007][Banaszewski, 2009][Gabbrielli, Martini, 2010][Roy e Haridi, 2004][Simão e Stadzisz, 2008, 2009].

Neste âmbito, uma alternativa é o Paradigma Orientado a Notificações (PON). O PON foi concebido a partir de uma teoria de Controle Discreto e Inferência [Simão, 2005][Simão e Stadzisz, 2008, 2009][Simão, Stadzisz e Tacla, 2009]. Ele se propõe a eliminar algumas das deficiências dos atuais paradigmas em relação a avaliações causais desnecessárias e acopladas, evitando o processo de inferência monolítico baseado em pesquisas por meio de um mecanismo baseado no relacionamento de entidades computacionais notificantes [Banaszewski et al., 2007][Banaszewski, 2009][Simão e Stadzisz, 2008, 2009].

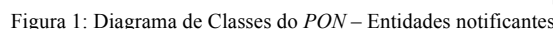
A *UML* fornece um conjunto rico de conceitos de modelagem e notações que foram desenvolvidos para satisfazer as necessidades de projetos típicos de software. No entanto, há casos em que são necessárias características adicionais de modelagem, além daquelas definidas pela *UML* padrão, como no caso de domínios particulares de aplicação (ex: tempo-real) ou implementação de tecnologias (ex: *CORBA*).

Neste contexto, este artigo apresenta um perfil *UML* por meio da especificação de mecanismos de extensão da *UML* - como estereótipos, valores etiquetados e restrições - que definem os principais conceitos do PON, melhor adequando sua modelagem em *UML*. O perfil *UML* criado para o PON é denominado *Perfil PON (NOP Profile* em inglês) e particulariza o metamodelo da *UML* a fim de suprir as necessidades específicas deste novo paradigma de programação.

apresenta o Paradigma Orientado a Notificações (PON). A seção 3 descreve sucintamente a *UML* e seus mecanismos de extensão. A seção 4 apresenta o perfil *UML* criado para o PON. A seção 5 ilustra um exemplo de uso do perfil *UML* desenvolvido. E, por fim, na seção 6 são apresentadas as conclusões do trabalho.

O PON encontra inspirações no PI, tais como a flexibilidade algorítmica e a abstração em forma de classes/objetos da POO. O PON também aproveita conceitos próprios do PD, como facilidade de programação em alto nível e a representação do conhecimento em regras dos SBR. Assim, o PON provê a possibilidade de uso (de parte) de ambos os estilos de programação em seu modelo, ainda que os evolua e mesmo os revolucione (de certa maneira) no tocante ao processo de inferência ou cálculo lógico-causal [Simão e Stadzisz, 2008, 2009][Banaszewski, 2009].

Mais precisamente, o PON possui objetos que tratam dos elementos da base de fatos, que são genericamente modelados pela classe *FBE (Fact Base Element)*, conforme modelado na Figura 1. Cada *FBE* trata de seus atributos por meio de objetos da classe *Attribute* e seus serviços por meio de objetos da classe *Method*. Os objetos *FBE*, por meio de seus *Attributes* e *Methods*, são passíveis de correlação causal por meio de *Rules*, as quais se constituem em elementos fundamentais do PON.



visualização de que cada *Rule* é composta de outros objetos colaboradores. Por exemplo, a *Rule* apresentada –

RuleOpeningGate – tem um objeto *Condition* e um objeto *Action*. A *Condition* trata da decisão da *Rule*, enquanto a *Action* trata da execução das ações da *Rule*. Assim sendo,

Condition e *Action* trabalham para realizar o conhecimento lógico e causal da *Rule*.

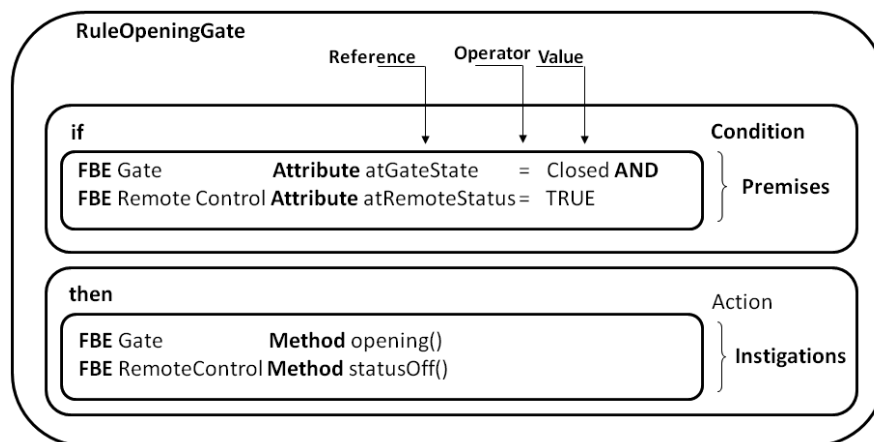


Figura 2: Exemplo de uma *Rule* em PON

A *Rule* da Figura 2 é referente a um fragmento de um sistema que simula um portão eletrônico. A *Condition* desta *Rule* lida com a decisão de abertura do portão (*Gate*) caso o controle remoto (*RemoteControl*) tenha sido pressionado. Esta *Condition* é composta por duas *Premises* que se constituem em outro tipo de objeto-inteligente. Estas duas *Premises* fazem as seguintes verificações sobre as *FBEs* *Gate* e *RemoteControl*: a) o portão eletrônico está fechado? (*atGate = Closed*) e b) o controle remoto foi pressionado? (*atRemoteStatus=TRUE*). Assim, conclui-se (em geral) que os estados dos atributos das *FBEs* compõem os fatos a serem avaliados pelas *Premises*.

Na verdade, cada *Premise* avalia o estado de um (ou mesmo dois) *Attributes* de *FBE*. De fato, para cada mudança de estado de um *Attribute* da *FBE*, ocorrem automaticamente avaliações (lógicas) somente nas *Premises* relacionadas com eventuais mudanças nos seus estados. Igualmente, a partir da mudança de estado das *Premises*, ocorrem automaticamente avaliações (causais) somente nas *Conditions* relacionadas com eventuais mudanças de seus estados. Entretanto, isto tudo se dá por meio de uma cadeia de notificações entre objetos inteligentes, o que se constitui no fundamento do PON conforme modelado na Figura 1.

A Figura 2 ilustra que cada *Premise* possui três elementos principais que colaboram entre si para a sua avaliação:

- *Reference*: guarda referência a um objeto *Attribute*, que por sua vez notifica o objeto *Premise* quando ocorre mudança em seu estado (ex: *atGateState* e *atRemoteStatus*).
- *Operator*: operador usado para fazer comparações entre os valores *Reference* e *Value*. Em PON um operador lógico pode ser dos tipos: *DIFFERENT* (diferente), *GREATERTHAN* (maior que), *SMALLERTHAN* (menor que), *GREATEROREQUAL* (maior ou igual) e *SMALLEROREQUAL* (menor ou igual).

- *Value*: pode ser uma constante ou referência para outro objeto *Attribute* (ex: *Closed* e *TRUE*).

As *Premises* relacionam-se entre si por meio de operadores lógicos definidas pelo objeto *Condition*. Um operador lógico pode ser do tipo *Conjunção* (*Conjunction*), *Disjunção* (*Disjunction*) ou sem operador lógico no caso em que há somente uma premissa (*Single*). Na Figura 2 o operador lógico da condição é de *conjunção* (*AND*).

Em suma, cada *Attribute* notifica as *Premises* relevantes sobre seus estados somente quando se fizer efetivamente necessário. Cada *Premise*, por sua vez, notifica as *Conditions* relevantes dos seus estados usando o mesmo princípio. Baseado nestes estados notificados é que a *Condition* pode ser aprovada ou não. Se a *Condition* é aprovada, a respectiva *Rule* pode ser ativada executando sua *Action* [BANASZEWSKI, 2009].

Por sua vez, uma *Action* também é um objeto inteligente que se conecta a objetos inteligentes de outro tipo, as *Instigations*. Neste exemplo, a *Action* contém duas *Instigations* que fazem com que: a) o portão passe para o estado “abrindo” (executando o método *opening()*) e b) configura o estado do controle remoto para desligado (executando o método *statusOff()*). Efetivamente, o que uma *Instigation* faz é instigar um ou mais métodos responsáveis por realizar serviços ou habilidades de uma *FBE*.

Cada método de um *FBE* é também tratado por um objeto inteligente, chamado de *Method*. A execução de um *Method* geralmente muda o estado de um ou mais *Attributes*. Na verdade, os conceitos de *Attribute* e *Method* representam uma evolução dos conceitos de atributos e métodos de classe da OO. A diferença é o desacoplamento explícito da classe proprietária e a “inteligência” colaborativa pontual para com *Premises* e *Instigations* (Simão e Stadzisz, 2008). Portanto, os objetos *Methods* encapsulam a funcionalidade de um método que pode ser usado de duas formas: guardar referência para a chamada de um método de um objeto

(*MethodDerived*) ou implementar o método em seu próprio escopo (*MethodPointer*).

Considerando-se isto, nota-se que a essência da computação no PON está organizada e distribuída entre entidades autônomas e reativas que colaboram por meio de notificações pontuais. Este arranjo forma o mecanismo de notificações, o qual determina o fluxo de execução das aplicações. Por meio deste mecanismo, as responsabilidades de um programa são divididas entre os objetos do modelo, o que permite execução otimizada e ‘desacoplada’ (ou minimamente acoplada) útil para aproveitamento correto de mono-processamento, bem como o processamento distribuído.

Outrossim, o PON atualmente está materializado na forma de um *framework/wizard* em C++ enquanto uma linguagem e compilador PON permanecem como um trabalho futuro. Muito embora um paradigma possa se materializar em outro (como um programa POO materializado em uma linguagem procedimental) e isto seja natural em paradigmas emergentes, seria mais

confortável e apropriado um ambiente efetivamente voltado para o PON [Banaszewski, 2009].

Este artigo se baseia no *framework* descrito em Banaszewski [2009] que contempla o PON implementado para ambientes monoprocessados. A estrutura básica do *framework* do PON é composta por dois pacotes de classes principais: *Core* e *Application*. As responsabilidades desses pacotes são as seguintes:

- *Core*: contém as classes que modelam os objetos colaboradores do PON que participam do mecanismo de notificações.
- *Application*: permite a definição de uma nova aplicação em PON por meio da extensão de sua classe denominada *Application*, que permite que a nova aplicação utilize os serviços do pacote *core*. Esses dois pacotes possuem uma relação de importação entre si, o que significa que, a aplicação que deriva da classe *Application* automaticamente estará importando as classes do pacote *core*. A Figura 3 ilustra o pacote *Application* e as classes que o compõem.

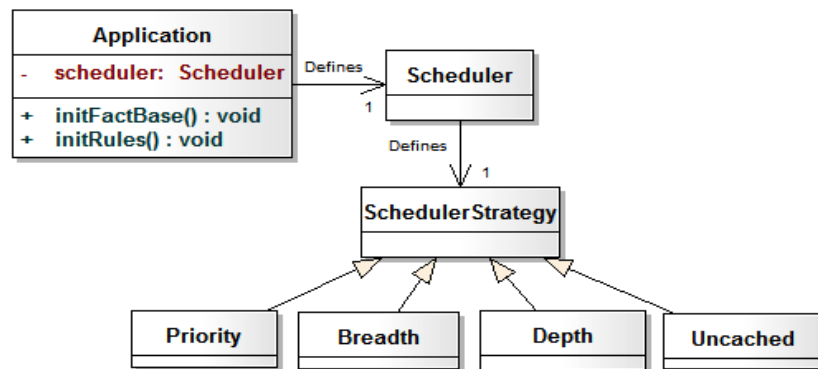


Figura 3: Diagrama de Classes - *Application*

Este pacote define a classe *Application* que possui dois métodos principais – *initFactBase* e *initRules* – usados para definir os elementos *FBE* e *Rule* respectivamente. Adicionalmente, a classe *Application* possui um atributo *scheduler* que determina a estratégia de resolução de conflitos de aprovação das regras, que pode ser de cinco tipos: *BREADTH* (se baseia no escalonamento FIFO), *PRIORITY* (organiza os objetos *Rules* de acordo com as prioridades definidas nos mesmos), *DEPTH* (se baseia no escalonamento FILO), *UNCACHED* (a regra é adicionada ao escalonador e aprovada imediatamente) e *NO_ONE* (não há estratégia definida e a regra não é enviada ao escalonador, mas é aprovada imediatamente).

3. UML e mecanismos de extensão

A Linguagem de Modelagem Unificada (*UML*) é definida como uma linguagem para especificação, visualização, construção e documentação de artefatos de sistemas de *software* por meio do uso do Paradigma Orientado a Objetos [OMG, 2003][GUEDES, 2004]. A arquitetura da *UML* é baseada em uma estrutura de metamodelos de quatro camadas definida pela *OMG*, que separa os diferentes níveis conceituais para construir um modelo. As camadas são: camada de informação M0 (define as

instâncias do sistema), camada de modelo M1 (define os modelos que descrevem as instâncias da camada M0), camada de metamodelo M2 (compreende a descrição da estrutura e semântica do modelo definido da camada M1) e camada de meta-metamodelo M3 (define os conceitos, estrutura e semântica que são utilizados para definir os metamodelos). A *UML* é definida por seu metamodelo que reside na camada M2 dessa arquitetura, sendo descrita por uma combinação de notação gráfica, linguagem natural e linguagem formal [OMG, 2003].

A *UML* permite que seu metamodelo seja estendido por meio de seus mecanismos de extensão, que particularizam o metamodelo para domínios de aplicações específicos, fornecendo novas sintaxe e semântica aos seus elementos. Um conjunto desses elementos, agrupados dentro de um pacote, formam um Perfil *UML*. Esses mecanismos permitem a adição de novos elementos de modelagem ao metamodelo da *UML*. Os mecanismos de extensão da *UML* são três [Lima, 2008]:

- Estereótipo: permite a classificação de um elemento de modelo de acordo com um elemento de modelo base já existente na *UML*. Estereótipos devem possuir restrições e valores etiquetados para adicionar informações necessárias aos novos

elementos (ex: *idClass* na Figura 4).

- Valor etiquetado: permite explicitar uma propriedade de um elemento, sendo que essas informações podem ser adicionadas a qualquer elemento do modelo. No metamodelo, os valores etiquetados são representados como atributos da classe que define um estereótipo (ex: *identificador* na Figura 4), mas no modelo são apresentadas em um novo compartimento dos elementos, denominado “tags”.

- Restrição: é uma informação semântica anexada a um ou mais elementos do modelo para expressar uma condição que o sistema deve satisfazer. Tal especificação é escrita em uma determinada linguagem de restrições, que pode ser mais ‘formal’ (ex: *OCL* - *Object Constraint Language*) ou menos formal (ex: texto em português). Ex: “*identificador* > 0” na Figura 4.

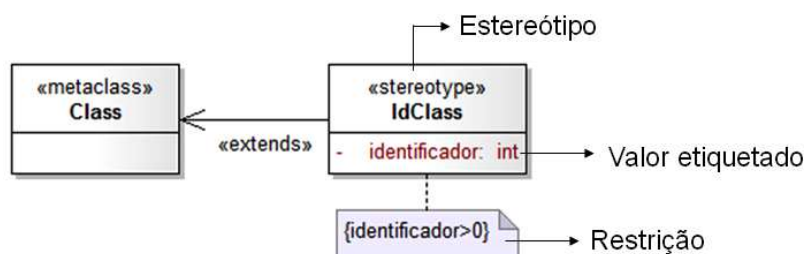


Figura 4: Exemplo de mecanismos de extensão no metamodelo

4. Perfil UML para o PON

O perfil *UML* para o PON estende o metamodelo da *UML* e foi elaborado baseado no método de criação de perfis *UML* proposto por Fuentes e Vallecillo [2004] e nos perfis *UML* reconhecidos pela *OMG*, como o *UML Profile for CORBA Components* [OMG, 2005a] e o *UML Profile for Schedulability, Performance, and Time Specification* [OMG, 2005b]. Todas essas fontes concordam que a construção de um perfil *UML* envolve a criação de dois artefatos: o modelo do domínio (ou metamodelo) e o perfil *UML* propriamente dito. O modelo do domínio descreve o conjunto de elementos que compreendem o sistema a ser definido pelo perfil *UML* e o relacionamento entre eles, permitindo identificar seus conceitos relevantes e restrições. Uma vez identificado o modelo do domínio o próximo passo é a criação do perfil *UML*. A seção 4.1 descreve o modelo do domínio do PON e a seção 4.2 apresenta o perfil *UML* criado, denominado *NOP Profile* (Perfil PON).

4.1. Modelo do domínio

A identificação dos conceitos do domínio da aplicação é o estágio inicial do processo de desenvolvimento de um perfil *UML*. No modelo do domínio deve-se adicionar as definições das entidades do domínio, o relacionamento entre elas e as restrições que controlam a estrutura e o comportamento dessas entidades. Um modelo conceitual do domínio pode ser construído usando-se os elementos de modelagem da *UML* como classes, pacotes e associações. Neste caso, os conceitos do domínio são representados pelas classes e os atributos são utilizados para capturar as propriedades desses conceitos, sendo que

as relações entre os conceitos são representados por associações. No âmbito deste trabalho, o modelo do domínio da aplicação foi elaborado utilizando-se o diagrama de classes da *UML*, sendo composto pelos dois pacotes de classes do *framework* do PON: *Core* e *Application*. Por meio desses diagramas foi possível identificar o significado e o uso dos elementos, relações e restrições do PON. Os modelos do domínio do PON são ilustrados pelas Figuras 1 e 3 da seção 2.

4.2. NOP profile

Após a identificação dos modelos do domínio do PON, o próximo passo foi a criação do perfil *UML* para o PON, denominado o *NOP Profile*. O *NOP Profile* é composto por um pacote *UML* estereotipado <<profile>> com mesmo nome do perfil, que compreende outros dois pacotes – *NOP Profile Core* e *NOP Profile Application* – um para cada um dos pacotes de classes do *framework* do PON.

NOP Profile - Core. O pacote *NOP Profile – Core* é composto por elementos de extensão da *UML* (estereótipos, valores etiquetados e restrições) que representam e descrevem os objetos participantes do mecanismo de notificações do PON. Esses elementos de extensão foram obtidos a partir da análise do modelo do domínio da Figura 1 (que ilustra o diagrama de classes do *framework core*). Primeiramente, foi criado um estereótipo para cada elemento relevante definido no modelo do domínio e, em seguida, foram associados esses estereótipos aos elementos do metamodelo da *UML* por meio do relacionamento de extensão (<<extends>>). A Figura 5 exhibe o perfil *NOP Profile – Core* criado.

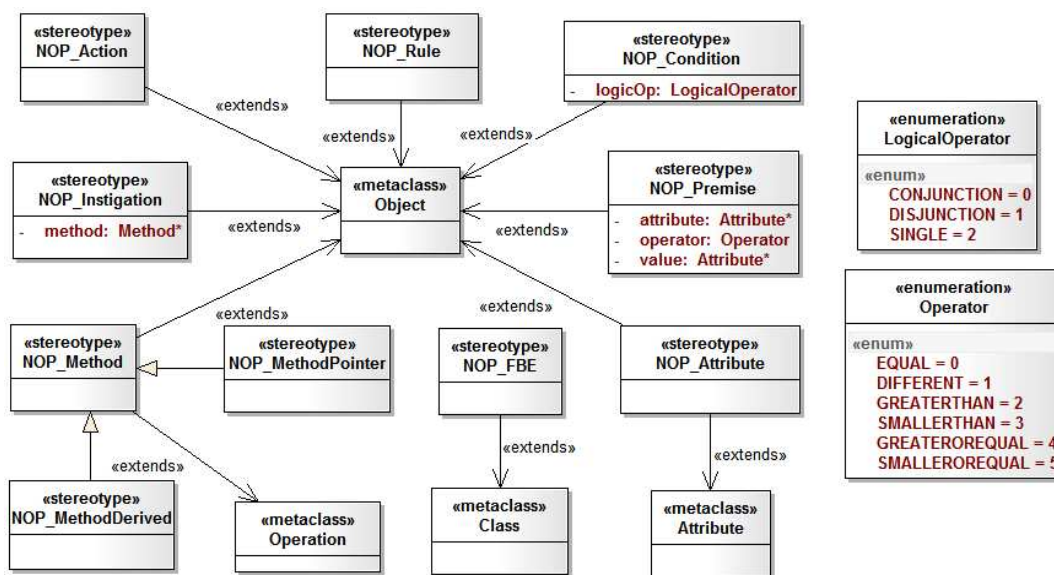


Figura 5: NOP Profile – Pacote Core

Antes de apresentar os estereótipos criados nestes pacotes é importante ressaltar alguns conceitos do PON. Quando o programador cria um novo programa em PON, os elementos *Rule* e os elementos colaboradores – *Attribute*, *Premise*, *Condition*, *Action*, *Instigation* e *Method* - são expressos como instâncias das respectivas classes no *framework*, ou seja, o programador não cria classes para esses elementos, apenas objetos. Por sua vez novos elementos *FBEs* são criados pelo programador como uma subclasse da classe *FBE* do *framework*, adicionando assim um novo elemento de dados à base de fatos.

Ao criar um novo *FBE*, o programador define adicionalmente seus métodos e atributos como sendo dos tipos das classes *Method* e *Attribute* do *framework* do PON. Quando instanciados os *FBE*, juntamente com seus *Methods* e *Attributes*, esses dois últimos entram na cadeia de notificação do PON uma vez adicionados às premissas (*Premises*) e instigações (*Instigations*) das regras (*Rules*). Portanto, os *Methods* e *Attributes* são, respectivamente, métodos e atributos do ponto de vista de um *FBE*, mas também são objetos do ponto de vista do mecanismo de notificação do PON.

Para cada elemento do mecanismo de notificações do PON – *Rule*, *Attribute*, *Premise*, *Condition*, *Action*, *Instigation* e *Method* – foram criados estereótipos –

NOP_Rule, *NOP_Attribute*, *NOP_Premise*, *NOP_Condition*, *NOP_Action*, *NOP_Instigation* e *NOP_Method* respectivamente – que estendem a metaclasses *Object*, uma vez que todos esses elementos representam instâncias das classes correspondentes do *framework* do PON. Também foram criados os estereótipos *NOP_MethodPointer* e *NOP_MethodDerived* que estendem o estereótipo *NOP_Method*. Para o elemento *FBE* foi criado um estereótipo – *NOP_FBE* – que estende a metaclasses *Class*, adicionalmente, como uma subclasse *FBE* é composta por métodos e atributos, os estereótipos *NOP_Attribute* e *NOP_Method* também estendem as metaclasses *Attribute* e *Operation*, respectivamente.

Além desses estereótipos foram criadas duas enumerações: *LogicalOperator* e *Operator*. A primeira define os tipos que um operador lógico pode assumir: *SINGLE*, *CONJUNCTION* e *DISJUNCTION*, e a segunda define os operadores de comparação possíveis PON, que são eles: *EQUAL*, *DIFFERENT*, *GREATERTHAN*, *SMALLERTHAN*, *GREATEROREQUAL*, *SMALLEROREQUAL*. Adicionalmente foram criados, no pacote *Core*, estereótipos que estendem a metaclasses *Association* e definem os relacionamentos entre os estereótipos da Figura 5. A Figura 6 ilustra esses estereótipos de associação.

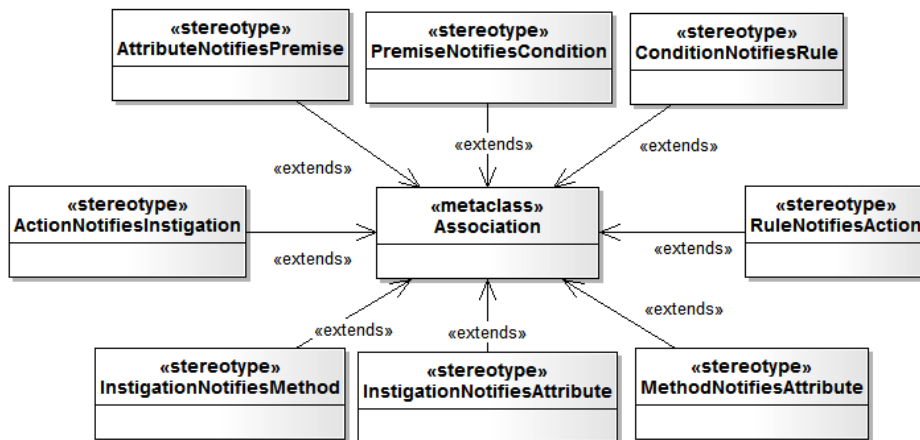


Figura 6: NOP Profile – Pacote Core - Relacionamentos

A Tabela 1 apresenta os estereótipos básicos do pacote Core e suas descrições, sendo que “Classe UML Base” representa o elemento do metamodelo da UML que serve de base para o estereótipo. O estereótipo `<<Rule>>`

mostra um exemplo de restrição em linguagem portuguesa e OCL. Por sua vez, a Tabela 2 descreve os estereótipos de relacionamento e suas restrições.

Estereótipo	Classe UML Base	Descrição e OCL
<code><<NOP_FBE>></code>	Class	Representa elementos da base de fatos (ex: entidades do mundo real) de um problema computacional e descreve os estados e os serviços dessas entidades por meio de seus atributos e métodos.
<code><<NOP_Rule>></code>	Object	Gerencia o conhecimento causal sobre qualquer comportamento do sistema, representando as expressões causais desse sistema. Restrições: um estereótipo <i>Rule</i> só pode associar-se com os estereótipos <code><<Condition>></code> e <code><<Action>></code> . context Rule inv: self.allOppositeAssociationEnds -> forAll (participant.isStereotyped('Condition') or participant.isStereotyped('Action')).
<code><<NOP_Attribute>></code>	Object, Attribute	Representa os atributos de um <i>NOP_FBE</i> . É responsável por reagir a mudanças em seus estados notificando os objetos <code><<NOP_Premise>></code> pertinentes.
<code><<NOP_Premise>></code>	Object	Realiza o teste lógico sobre os estados dos objetos <code><<NOP_FBE>></code> . Possui três valores etiquetados: attribute (guarda referência a um objeto <i>Attribute</i> , que notifica o objeto <i>Premise</i> quando ocorre mudança em seu estado); operator do tipo Operator (operador lógico usado para fazer comparações entre os valores <i>attribute</i> e <i>value</i>) e value (pode ser uma constante ou uma referência para outro objeto <i>Attribute</i>).
<code><<NOP_Condition>></code>	Object	Responsável pela parte de decisão de um objeto <code><<NOP_Rule>></code> . Possui um valor etiquetado logicOp do tipo <i>LogicalOperator</i> que define o operador lógico da condição a ser aplicado entre as premissas.
<code><<NOP_Action>></code>	Object	Responsável pela parte de ação de uma regra, sendo responsável por iniciar a cadeia de execução dos métodos dos objetos <code><<NOP_FBE>></code> .
<code><<NOP_Instigation>></code>	Object	Responsável por invocar os serviços de um objeto <code><<NOP_Method>></code> ou <code><<NOP_FBE>></code> diretamente. Possui um valor etiquetado: method (Guarda referência a um objeto <i>Method</i> a ser executado por um <i>Instigation</i>).
<code><<NOP_Method>></code>	Object, Operation	Responsável por receber notificações de objetos <code><<NOP_Instigation>></code> de uma regra gerando a sua execução. Os objetos <code><<NOP_Method>></code> encapsulam a funcionalidade de um método que pode ser de dois tipos: <code><<NOP_MethodDerived>></code> (guarda referência para a chamada de um método de um objeto) e <code><<NOP_MethodPointer>></code> (guarda referência para a chamada de um método de um objeto implementando no escopo do <i>FBE</i>).

Table 1: Estereótipos do pacote Core

Estereótipo	Descrição
<<AttributeNotifiesPremise>>	Relacionamento de notificação entre os estereótipos <<NOP_Attribute>> e <<NOP_Premise>> que ocorre no sentido <i>NOP_Attribute</i> → <i>NOP_Premise</i> . Um objeto <<NOP_Attribute>> pode notificar um ou mais objetos <<NOP_Premise>>. A fim de representar os diagramas com um nível de abstração maior, este relacionamento também pode ocorrer entre <<NOP_FBE>> e <<NOP_Premise>> indicando que, apesar do <<NOP_Attribute>> estar omitido do diagrama, subentende-se que uma notificação ocorreu entre um <i>Attribute</i> do <i>FBE</i> e um <i>Premise</i> (como utilizado na Figura 9 da seção 5 entre <i>objRemoteControl</i> e <i>pre1_rlOpening</i>).
<<PremiseNotifiesCondition>>	Relacionamento de notificação entre os estereótipos <<NOP_Premise>> e <<NOP_Condition>> que ocorre no sentido <i>NOP_Premise</i> → <i>NOP_Condition</i> . Um objeto <<NOP_Premise>> pode notificar um ou mais objetos <<NOP_Condition>>.
<<ConditionNotifiesRule>>	Relacionamento de notificação entre os estereótipos <<NOP_Condition>> e <<NOP_Rule>> que ocorre no sentido <i>NOP_Condition</i> → <i>NOP_Rule</i> . Um objeto <<NOP_Condition>> pode notificar um objeto <<NOP_Rule>>.
<<RuleNotifiesAction>>	Relacionamento de notificação entre os estereótipos <<NOP_Rule>> e <<NOP_Action>> que ocorre no sentido <i>NOP_Rule</i> → <i>NOP_Action</i> . Um objeto <<NOP_Rule>> pode notificar um objeto <<NOP_Action>>.
<<ActionNotifiesInstigation>>	Relacionamento de notificação entre os estereótipos <<NOP_Action>> e <<NOP_Instigation>> que ocorre no sentido <i>NOP_Action</i> → <i>NOP_Instigation</i> . Um objeto <<NOP_Action>> pode notificar um ou mais objetos <<NOP_Instigation>>.
<<InstigationNotifiesMethod>>	Relacionamento de notificação entre os estereótipos <<NOP_Instigation>> e <<NOP_Method>> que ocorre no sentido <i>NOP_Instigation</i> → <i>NOP_Method</i> . Um objeto <<NOP_Instigation>> pode notificar zero ou um objeto <<NOP_Method>>. A fim de representar os diagramas com um nível de abstração maior, este relacionamento também pode ocorrer entre <<NOP_Instigation>> e <<NOP_FBE>> indicando que, apesar do <<NOP_Method>> e <<NOP_Attribute>> estarem omitidos do diagrama, subentende-se que uma notificação ocorreu entre um <i>Instigation</i> e um <i>Method</i> e, na sequência, entre um <i>Method</i> e um <i>Attribute</i> do <i>FBE</i> (como utilizado na Figura 9 da seção 5 entre <i>inst1_rlOpening</i> e <i>objRemoteControl</i>).
<<InstigationNotifiesAttribute>>	Relacionamento de notificação entre os estereótipos <<NOP_Instigation>> e <<NOP_Attribute>> que ocorre no sentido <i>NOP_Instigation</i> → <i>NOP_Attribute</i> . Um objeto <<NOP_Instigation>> pode notificar zero ou um objeto <<NOP_Attribute>>. A fim de representar os diagramas com um nível de abstração maior, este relacionamento também pode ocorrer entre <<NOP_Instigation>> e <<NOP_FBE>> indicando que, apesar do <<NOP_Attribute>> estar omitido do diagrama, subentende-se que uma notificação ocorreu entre um <i>Instigation</i> e um <i>Attribute</i> do <i>FBE</i> (como utilizado na Figura 9 da seção 5 entre <i>inst2_rlOpened</i> e <i>objGate</i>).
<<MethodNotifiesAttribute>>	Relacionamento de notificação entre os estereótipos <<NOP_Method>> e <<NOP_Attribute>> que ocorre no sentido <i>NOP_Method</i> → <i>NOP_Attribute</i> . Um objeto <<NOP_Method>> pode notificar zero ou mais objetos <<NOP_Attribute>>.

Table 2: NOP Profile – Pacote *Core* – Estereótipos que estendem a metaclass *Association*

NOP Profile - Application. Seguindo os mesmos passos para a criação do *NOP Profile Core*, foi construído o *NOP Profile Application* analisando-se o modelo de domínio da Figura 3. Neste perfil, a classe *Application* do modelo do domínio foi transformada em um estereótipo – *NOP_Application* – que estende a metaclass *Class* do metamodelo da UML, uma vez que uma nova aplicação em PON é representada como uma subclasse da classe *Application*. Já os métodos da classe *Application* do

framework do PON foram transformados em estereótipos que estendem a metaclass *Operation* da UML, sendo eles: *initFactBase* e *initRules*. Também foi criada uma enumeração – *SchedulerStrategy* – que define as estratégias de resolução de conflitos entre regras existentes em PON, que podem ser: *BREADTH*, *PRIORITY*, *DEPTH*, *UNCACHED* e *NO_ONE*. A Figura 7 ilustra o pacote de perfil *NOP Profile Application* criado.

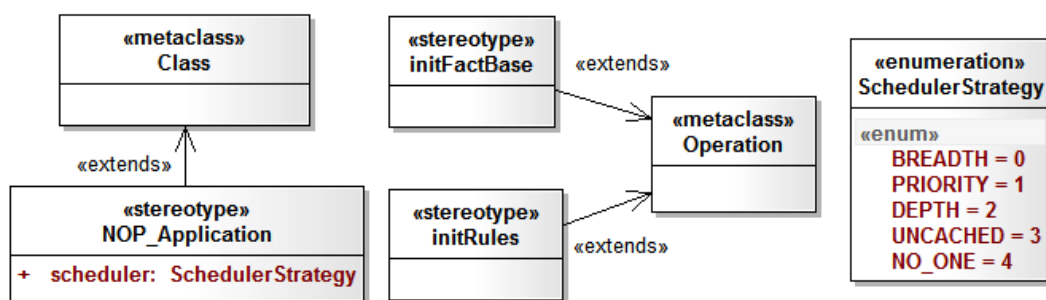


Figura 7: NOP Profile – Application

A Tabela 3 descreve os estereótipos do pacote *Application*.

Estereótipo	Classe UML Base	Descrição
<<NOP_Application>>	Class	Representa o ponto de extensão entre os dois pacotes do framework do PON (Core e Application). Possui um valor etiquetado <i>scheduler</i> do tipo <i>SchedulerStrategy</i> que define a estratégia de resolução de conflito do sistema.
<<initFactBase>>	Operation	Método responsável por instanciar os objetos <i>FBE</i> definidos na aplicação.
<<initRules>>	Operation	Método responsável por instanciar os objetos <i>Rule</i> definidos na aplicação.

Tabela 3: Estereótipos do pacote *Application*

5. Exemplo de uso

Esta seção apresenta um exemplo de aplicação do *NOP Profile* na modelagem de um simulador de Portão Eletrônico. Este sistema, de maneira resumida, possui dois casos de uso: “Abrir Portão” e “Fechar Portão”. No entanto, este exemplo de uso será focado no caso de uso

“Abrir Portão”. Basicamente, neste caso de uso, o portão começa fechado e, uma vez apertado a controle remoto, o portão começa a abrir. Após 30 segundos o portão estará aberto. A Figura 8 apresenta o diagrama de classes desse sistema aplicando-se o perfil *NOP Profile*.

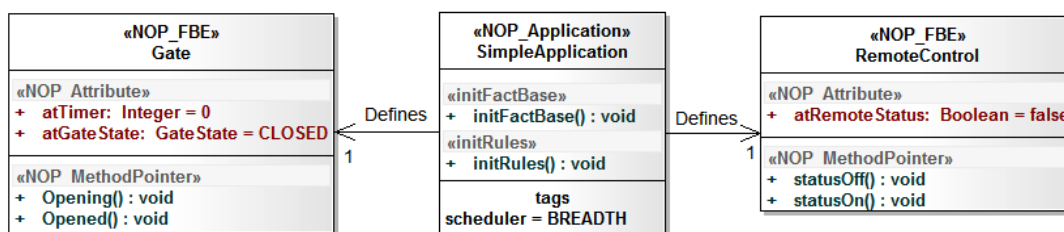


Figura 8: Exemplo Portão Eletrônico – Diagrama de Classes

Uma nova aplicação em PON é representada por uma subclasse da classe *Application* do *framework* do PON, que recebe o estereótipo <<NOP_Application>>. Uma nova aplicação obrigatoriamente possui os métodos *initFactBase* e *initRules* e uma estratégia de resolução de conflitos definida. Além disso, também é necessário criarem-se classes *FBEs* que representem os elementos da base de fatos do sistema.

No exemplo em questão, foi criada uma classe denominada *SimpleApplication* estereotipada <<NOP_Application>> e duas classes - *RemoteControle* e *Gate* - estereotipadas <<NOP_FBE>>. A classe *SimpleApplication* herdou o valor etiquetado “*scheduler*” (exibido no compartimento “*tags*”), que foi definido como “*BREATH*” para esta aplicação, e possui dois métodos estereotipados *initFactBase* e *initRules* que

possuem mesmo nome de seus estereótipos.

A classe *Gate* possui dois atributos - *atGateState* e *atTimer* - estereotipados <<NOP_Attribute>> além de dois métodos - *Opening* e *Opened* - estereotipados <<MethodPointer>>. O método *Opening* atribui “OPENING” ao atributo *atGateState* e inicia o incremento do valor do contador *atTimer*, e o método *Opened* atribui “OPENED” a *atGateState* e zero ao contador *atTimer*. Por sua vez, a classe *RemoteControl* possui um atributo *atRemoteState*, estereotipado <<NOP_Attribute>> e dois métodos - *statusOn* e *statusOff* - estereotipados <<MethodPointer>>. O método *statusOn* atribui “TRUE” para o atributo *atRemoteStatus* e o método *statusOff* atribui “FALSE” para o mesmo atributo.

Uma vez definida a aplicação e os *FBEs*, é necessário modelar as *Rules* do sistema. A fim de facilitar o

entendimento, as regras do caso de uso “Abrir Portão” são apresentadas na Tabela 4.

Rules	Comditions com suas Premises	Actions com suas Instigations
rlOpeningGate	objRemoteControl.atRemoteStatus == true && objGate.atGateState == CLOSED && objGate.atTimer == 0	objRemoteControl->statusOff() && objGate->Opening()
rlOpenedGate	objGate.atGateState == OPENING && objGate.atTimer > 29	objGate->Opened() && objGate.atTimer=0

Tabela 4: Lista de regras

A Figura 9 ilustra o diagrama de objetos do caso de uso “Abrir Portão”. Aplicando-se o *NOP Profile* em um diagrama de objetos é possível modelar a composição estática das *Rules* em PON com a definição de suas premissas e instigações, além da definição das propriedades desses elementos por meio dos valores etiquetados. O diagrama de objetos do exemplo modela as duas regras da Tabela 4. A fim de exemplificação será descrita a regra *rlOpeningGate*. Esta regra recebe o estereótipo <<NOP_Rule>> e avalia os estados dos objetos *objGate* e *objRemoteControl*, objetos estes obtidos das respectivas classes *Gate* e *RemoteControl* definidas na Figura 8. A *Rule* em questão é composta por um objeto *Condition* – *condition_rlOpening* – e um objeto *Action* – *action_rlOpening*.

O objeto *Condition_rlOpeningGate*, recebe o estereótipo <<NOP_Condition>> e possui três premissas estereotipadas <<NOP_Premise>>: *pre1_rlOpening*, *pre2_rlOpening* e *pre3_rlOpening*, que por sua vez avaliam os estados dos objetos *FBEs*. Por meio dos seus valores etiquetados definidos no *NOP Profile* – *attribute*, *operator* e *value* – e, que aparecem no compartimento “tags” desses objetos, a premissa *pre1_rlOpening* testa se o atributo *atRemoteStatus* do *objRemoteControl* é igual a *true*. Por sua vez a premissa *pre2_rlOpening* testa se o atributo *atTimer* do *objGate* é igual a zero e, por fim, a premissa *pre3_rlOpening* testa *atGateState* do *objGate* é igual a *CLOSED*. Estas três premissas estão ligadas por um operador lógico de Conjunção (*CONJUNCTION*) definido pelo valor etiquetado *logicOp* no objeto *condition_rlOpening*.

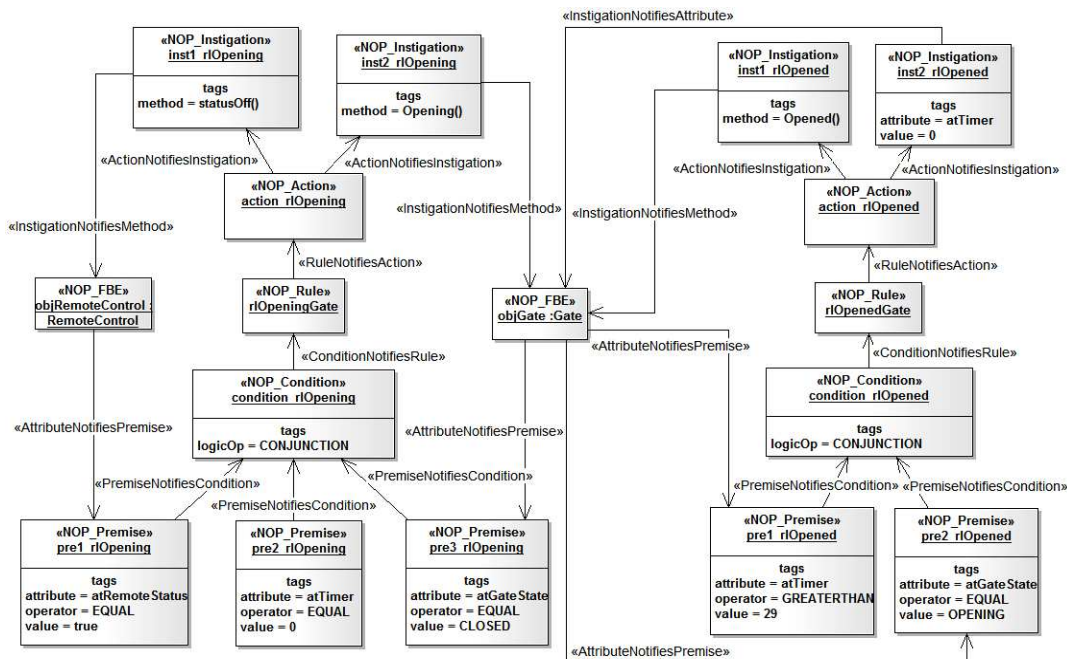


Figura 9: Exemplo Portão Eletrônico – Diagrama de Objetos

A ação da regra - *action_rlOpening*- recebe o estereótipo <<NOP_Action>> e é formada por duas instigações estereotipadas <<NOP_Instigation>>: *inst1_rlOpening* e *inst2_rlOpening*. Por meio também de seus valores etiquetados, no caso *method*, essas instigações definem o método a ser executado. No exemplo em questão, o objeto *inst1_rlOpening* executa o método “*statusOff()*” e

o objeto *inst2_rlOpening* executa o método “*Opening()*”. Todos esses objetos se relacionam por meio dos estereótipos de associação definidos no *NOP Profile* (Tabela 2).

Adicionalmente, este tipo de representação da estrutura estática de uma *Rule* pode ser mapeada para uma rede de Petri a fim de modelar a dinâmica de um sistema em

PON. Algumas vantagens das redes de Petri em relação aos diagramas de estados, conforme apresentado em Döll [2002], é que elas permitem a modelagem de concorrência, sincronização e compartilhamento de recursos em um sistema, além de ser possível realizar a validação do modelo por meio da sua simulação ou análise. Em Simão [2005] e Simão e Stadzisz [2009] é apresentada a compatibilidade entre o predecessor do PON e as redes de Petri, na qual cada regra é interpretada como uma *transição*, as condições e ações da regra são representados pelos *arcos*, e os estados dos atributos de um elemento são representados por *lugares* na rede.

Aproveitando essa interpretação, uma rede de Petri ordinária de um sistema em PON pode ser obtida a partir do diagrama de objetos apresentado na Figura 9 de acordo com o seguinte mapeamento: os objetos *NOP_Rule* do diagrama de objetos são substituídos por transições na rede de Petri; os atributos e seus estados que pertencem às premissas e às instigações são representados por *lugares* na rede; os objetos *NOP_Premise* são representados pelos arcos que chegam em uma transição que representa um *NOP_Rule*; e os *NOP_Instigation* são substituídos por arcos que saem de uma transição que também representa um *NOP_Rule*.

Conclusões

O Paradigma Orientado a Notificações (PON) tem sido materializado em termos de programação, mas não possuía ainda um método formalizado para orientar os desenvolvedores na elaboração de projetos de *software*. Portanto, como primeiro passo na construção deste método, o presente artigo apresentou o *NOP Profile*, um perfil UML criado para o PON. O *NOP Profile* tem por objetivo propor convenções na utilização da UML para modelagem de *software* neste novo paradigma. A aplicação do perfil UML foi ilustrada com exemplos envolvendo a modelagem de um simulador de Portão Eletrônico.

A extensão proposta é suportada pela UML e, portanto, pode ser facilmente integrada em ferramentas CASE que suportam a UML. O *NOP Profile* está sendo avaliado por meio da sua aplicação em diferentes projetos de *software*. Adicionalmente, os modelos UML criados, que utilizam o *NOP Profile*, podem ser verificados quanto à sua sintaxe por meio de funcionalidades padrões disponíveis nas ferramentas CASE que validam as restrições OCL definidas no perfil. Assim, foi apresentada uma primeira versão do *NOP Profile* que pode vir a ter evoluções conforme ele seja aplicado nos diferentes projetos e sejam identificadas novas necessidades.

Com base nos resultados produzidos, os autores julgam que o emprego de um Perfil UML para fins de metamodelagem não só é viável quanto representa uma abordagem muito prática e alinhada com a linguagem UML. O perfil criado possui grande valia na definição de programas em PON e visa sua aplicação em processos de projetos com esse paradigma de programação. Portanto, o próximo passo é a aplicação do *NOP Profile* em um processo de *software*. O objetivo é criar um método que será empregado em projetos de *software* baseados em PON. Para tanto, o *NOP Profile* deve ser aplicado nas

fases de modelagem, viabilizando uma modelagem consistente com os conceitos do *framework* do PON.

Referências bibliográficas

- [Banaszewski *et al.*, 2007] Banaszewski, R. F.; Stadzisz, P. C.; Tacla, C. A.; Simão, J. M. "Notification Oriented Paradigm (NOP): A software development approach based on artificial intelligence concepts," in *Proceedings of the VI Congress of LAPTEC*, Santos, Brazil, 2007.
- [Banaszewski, 2009] Banaszewski, R. F. "Paradigma Orientado a Notificações: Avanços e Comparações". Dissertação de Mestrado, CPGEI/UTFPR, Curitiba, 2009. Disponível em: http://arquivos.cpgei.ct.utfpr.edu.br/Ano_2009/dissertacoes/Dissertacao_500_2009.pdf.
- [Döll, 2002] Döll, L. M. (2002). Proposta de uma metodologia para a modelagem da dinâmica de sistemas orientados a objetos usando redes de petri predicado/transição. Dissertação de mestrado, CEFET-PR.
- [Fuentes e Vallecillo, 2004] Fuentes, L., Vallecillo, A. (2004). An Introduction to UML Profiles. The European Journal for the Informatics Professional, vol. 5, nº2.
- [Gabbrielli e Martini, 2010] Gabbrielli, M., Martini, S. "Programming Languages: Principles and Paradigms. Series: Undergraduate Topics in Computer Science". 1st Edition, 2010, XIX, 440 p., Softcover. ISBN: 978-1-84882-913-8.
- [Guedes, 2004] Guedes, G. T. (2004). *UML Uma abordagem Prática*. Novatec.
- [Lima, 2008] Lima, A. S. (2008). UML 2.0: do requisito à solução. São Paulo, Editora Érica, 3ªed.
- [OMG, 2003] OMG Unified Modeling Language Specification (2003). Version 1.5, OMG document formal/03-03-01.
- [OMG, 2005a] UML Profile for CORBA Components (2005). Version 1.0, OMG document formal/05-07-06.
- [OMG, 2005b] UML Profile for Schedulability, Performance, and Time Specification (2005). Version 1.1, OMG document formal/05-01-02.
- [Roy e Haridi, 2004] Roy, P. V., Haridi, S. "Concepts, Techniques, and Models of Computer Programming". MIT Press, 2004.
- [Simão, 2005] Simão, J. M. "A Contribution to the Development of a HMS Simulation Tool and Proposition of a Meta-Model for Holonic Control". Tese de Doutorado, CPGEI/UTFPR, Curitiba, 2005. Disponível em: http://arquivos.cpgei.ct.utfpr.edu.br/Ano_2005/teses/Tese_012_2005.pdf.
- [Simão e Stadzisz, 2008] Simão, J. M.; Stadzisz, P. C. "Paradigma Orientado a Notificações (PON) - Uma Técnica de Composição e Execução de Software Orientado a Notificações". Pedido de Patente submetida ao INPI/Brazil (Instituto Nacional de Propriedade Industrial) em 2008 e a Agência de Inovação/UTFPR em 2007. Nº INPI Provisório

015080004262. N° INPI Efetivo PI0805518-1. Patente submetida ao INPI. Brasil, 2008.

[Simão e Stadzisz, 2009] Simão, J. M.; Stadzisz, P. C. “Inference Process Based on Notifications: The Kernel of a Holonic Inference Meta-Model Applied to Control Issues”. *IEEE Transactions on Systems, Man and Cybernetics. Part A, Systems and Humans*, Vol. 39, Issue 1, 238-250, Digital Object Identifier 10.1109/TSMCA.2008.20066371, 2009.

[Simão, Stadzisz e Tacla, 2009] J. M. Simão, C. A. Tacla, and P. C. Stadzisz, “Holonic Control Meta-Model”. *IEEE Transactions on Systems Man & Cybernetics, Part A*. Vol. 39, NO. 5, September 2009 Pg. 1126-1139.

[Tanenbaum e Van Steen, 2002] Tanenbaum, A.S.; Van Steen, M. “Distributed Systems: Principles and Paradigms”, Prentice Hall, 2002.